React useEffect:-

React Hooks:-
1.State hooks
2.Context Hooks
3.Reference Hooks
4.Effect Hooks
5.Performance Hooks
6.Resource Hooks

1.State Hooks : Used to store and manage component data that changes over time.

2.Context Hooks : Used to share data globally across components without passing props manually.

3.Reference Hooks : Used to access DOM elements or store mutable values without re-rendering.

4.Effect Hooks : Used to handle side effects like API calls, timers, or lifecycle events.

5.Performance Hooks : Used to optimize components and prevent unnecessary re-renders.

6.Resource Hooks : Used to manage complex state logic or handle asynchronous updates efficiently.

What is the react useEffect?
useEffect is a React Hook that lets you perform side effects in functional components.

What does a side effect mean?

A side effect is something like:

- Fetching data from an API
- Updating the DOM manually
- Setting up timers (setTimeout, setInterval)
- Adding event listeners
- Logging data

Anything that happens outside normal rendering logic.

Syntax:-

```
import { useEffect } from "react";
```

```
useEffect(() => {
  // code to run
}, [dependencies]);
```

It has 2 parts:

1. A function (what to run)
2. A dependency array (when to run)

useContext Hook:

useContext is a React Hook that allows you to access data from a Context object without passing props manually through every component level.

It helps avoid prop drilling.

Syntax:

```
const value = useContext(ContextName);
```

Steps to Use useContext

Create Context using createContext()

Wrap components with Context.Provider

Use useContext() in child component

useReducer Hook:

Definition:

useReducer is a React Hook used for managing complex state logic in a structured way using a reducer function.

It is an alternative to useState.

Syntax:

```
const [state, dispatch] = useReducer(reducer, initialState);
```

When to Use useReducer Instead of useState?

Use useReducer when:

- State depends on previous state
- Multiple related state values exist
- Complex update logic
- Large forms
- Multiple actions on same state

UseEffectDemo.jsx

```
import { useState, useEffect } from "react";


function UseEffectDemo() {
  const [data, setData] = useState(0);


  useEffect(() => {
    setData(10);   // updating state after component mounts
  }, []);          // runs only once


  return (
    <h1>This is demo using React Hooks: {data}</h1>
  );
}


export default UseEffectDemo;
```

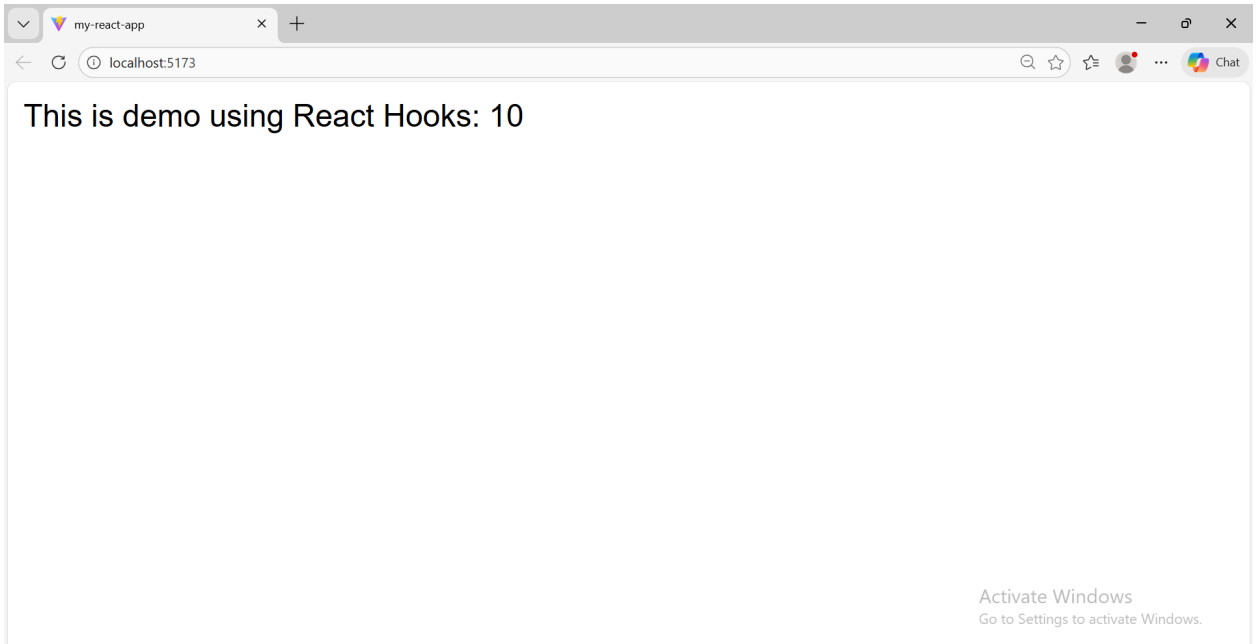
App.jsx

```
import { useState } from 'react'

import reactLogo from './assets/react.svg'

import viteLogo from '/vite.svg'

import './App.css'

import Promote from './components/Promote'

import UseEffectDemo from './components/UseEffectDemo'


function App() {

  const [age,setAge] = useState(20)

  const [phone,setPhone]=useState('123-456-7890')

  const [email,setEmail]=useState('abc@gmail.com')

  const [address,setAddress]=useState('123 Main St, Anytown, USA')


  return (

<>

    <div>

      {/* <Promote/> */}

      <UseEffectDemo/>

    </div>

</>

  )

}
```

```
export default App
```

Main.jsx

```
import { StrictMode } from 'react'

import { createRoot } from 'react-dom/client'

import './index.css'

import App from './App.jsx'


createRoot(document.getElementById('root')).render(

  <StrictMode>

    <App />

  </StrictMode>,

)
```



useEffect hook gets executed whenever the page renders

1. actions need to performed during the page load such initializing variables, or fething data from the cloud, etc.
2. run only once..... or run everytime the page renders

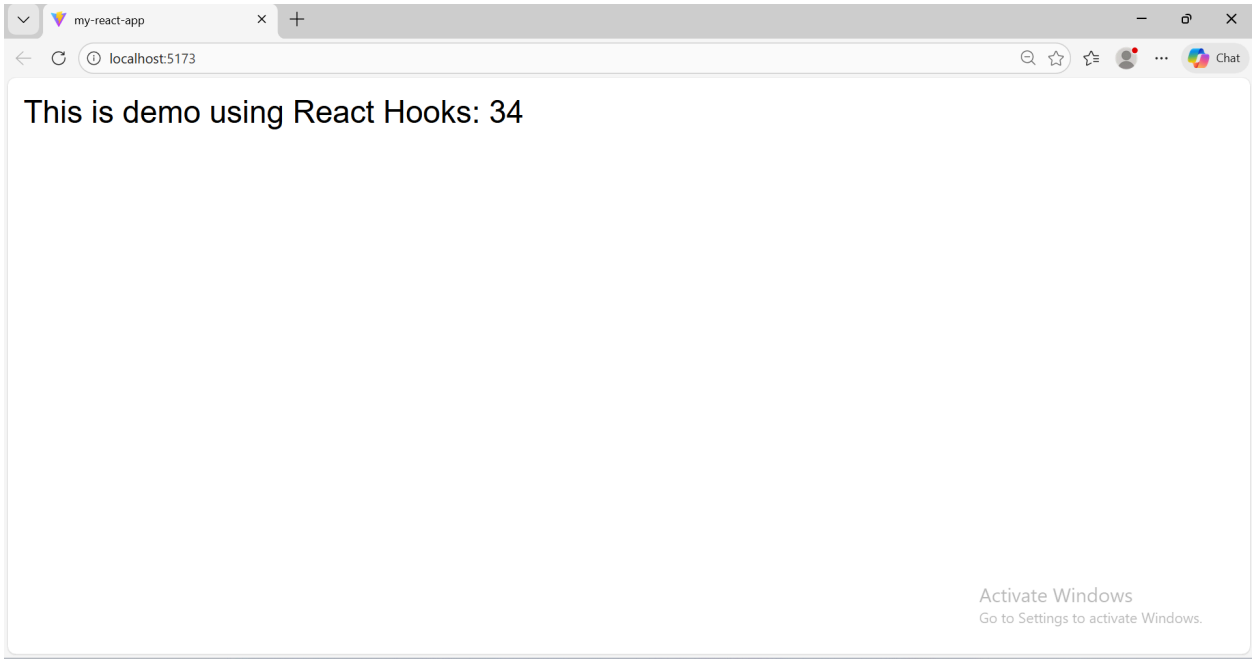
when the page gets loaded, it is called as page render

UseEffectDemo.jsx

```
import { useState, useEffect } from "react";


function UseEffectDemo() {

  const [count, setCount] = useState(-1);


  useEffect(() => {

    setTimeout(() => {
```

```
      setCount((count) => count + 1);
    }, 1000);
  });

  return (
    <h1>This is demo using React Hooks: {count}</h1>
  );
}

export default UseEffectDemo;
```

Count starts from -1 after every 1000 milliseconds it gets incremented(0,1,2……..,33,34,.....)



```
useEffect(() => {
   setTimeout(() => {
     setCount((count) => count + 1);
   }, 1000);
 },[]);
```

When [] is added the rendering gets paused why?

When you use []:

The effect runs only once

So state updates only once

So it looks like “it stopped”

But actually:

Rendering happens normally

The effect just doesn’t repeat

In vs code:-

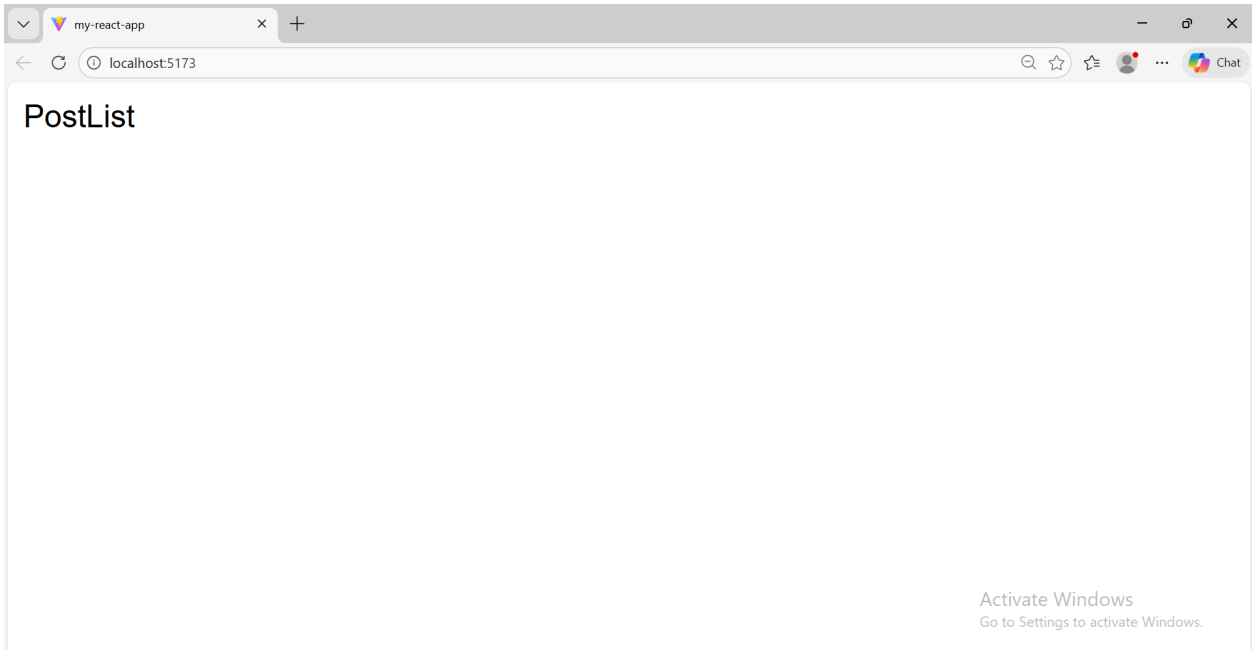
Go to extensions.
Search React
Find ES7+ React/Redux/React-Native snippets
Install it

Use of ES7+ React/Redux/React-Native snippets

It’s a VS Code extension that gives you shortcuts to quickly generate React/Redux code.

Instead of typing full boilerplate code, you type a shortcut and press Tab,it autogenerates

Type rfce ,u get the below boilerplate

```
import React from 'react'
```

```
function PostList() {
  return (
    <div>PostList</div>
  )
}

export default PostList
```